

PFDP

Mete Minbay (and Claude)

September 4, 2026

1 Problem definition

1.1 The objective

Fix the suffix array I of the text, and let $m = |I|$ denote the number of rows in I . For a word w , let $I_w = [\ell_w, r_w)$ denote the maximal LCP-interval in I such that every suffix in I_w starts with w . For some suffix s , let $\text{pos}(s)$ denote the actual position of s in the text. For some positive integer a , if $[\ell, r]$ is a row range of I with LCP-depth $a + 1$ where the $a + 1$ -th character is c , then a subset of the positions in I_c can be reconstructed in order by adding a shift of a to the positions in $[\ell, r]$. At a high level, we would like to prune I by finding rows whose positions in the text can be recreated by adding a constant offset to the positions of a contiguous window (in which case the former subset of rows can be deleted and replaced with a constant-size hash entry denoting "these rows are deleted; fetch the positions of this other range and add this offset").

Towards this goal, a **candidate** is a triple $k = (T_k, S_k, a_k)$: a target subset of rows $T_k \subseteq I_w$ of some word w , a contiguous source interval $S_k \subseteq I$, and a shift $a_k \in [a_{max}]$ such that the a_k -th suffix of the suffices in S_k reconstruct the suffices in T_k in order. In terms of compression, accepting a candidate k deletes rows T_k and reconstructs their positions via adding a shift a_k to every position stored in S_k . Let U denote the universe of all candidates, and $\text{cov}(k) = |T_k|$ denote the number of rows that are to be deleted when a candidate k is accepted. Under a subset of accepted candidates $X \subseteq U$, the resulting size of I becomes

$$m - \sum_{k \in X} \text{cov}(k)$$

Assuming that the space usage of our k -mer lookup structure increases monotonically with the number of remaining rows¹, the objective is to find a subset of consistent candidates that minimize the above expression.

1.2 The constraints

A candidate is valid if it satisfies the following conditions:

- **Source contiguity:** After the suffix array is compressed, constant-time lookup of words is achieved with a hash table that stores the *real* and the *referenced* occurrences of a word. References are stored as a 3-tuple (p, n, a) , indicating to read the positions from n consecutive rows starting from row p and to add a shift a to each of them. This structure cannot specify to "skip" specific rows, so all n rows must be part of the source ranks. In other words, S_k must be contiguous (for $i \in S_k : i = \min(S_k)$ or $i - 1 \in S_k$).

¹This assumption is not completely accurate: Since every accepted candidate increases the number of hash table entries, accepting many small-coverage candidates can cause the hash table index to take up extra space in a way that outweighs the space savings of the accepted candidates.

- **Target containment:** A target subset must be contained within the LCP-interval of a word (formally $\exists w : T_k \subseteq I_w$) so that the referenced occurrences of that word can be updated to recover the deleted row range if the candidate is accepted.

Additionally, accepted candidates are subject to the following constraints:

- **One reference per word:** Every word can have at most one hash table entry storing its referenced occurrences. Therefore, candidates k and k' cannot be accepted simultaneously if $T_k, T_{k'} \subseteq I_w$ for some w .
- **Source integrity:** If a row range T_k has been deleted to reference another range S_k , the source range S_k cannot be deleted. Therefore, candidates k and k' cannot be accepted simultaneously if $S_k \cap T_{k'} \neq \emptyset$ (notice that this already rules out self-referencing candidates, specifically k such that $S_k \cap T_k \neq \emptyset$).

1.3 Maximum weighted independent set

Consider a graph G where the vertices are the candidates and two candidates are connected if they cannot be accepted together due to the listed constraints, formally $V(G) = U$ and $E(G) = \{(k, k') : \exists w(T_k, T_{k'} \subseteq I_w) \vee S_k \cap T_{k'} \neq \emptyset\}$. Let the weight of each vertex be $w(k) := \text{cov}(k)$. Maximizing the number of deleted rows is equivalent to finding a maximum weighted independent set (MWIS) of G .

2 The method

2.1 Overview

Finding a MWIS admits a polynomial time dynamic programming algorithm on trees, but is NP-hard on general graphs. It is easy to see that the conflict graphs of our instances are not trees: Since every word can have ≤ 1 accepted candidate, multiple candidates belonging to the same word are already modeled as a clique. Solving the problem exactly is therefore unlikely unless the LCP-intervals give the graph some structure to be exploited.

The main idea of **pseudoforest-dp** is to heuristically sparsify the candidate space, then exactly solve the problem on the remaining candidates using dynamic programming.

2.2 Prerequisite: MWIS on trees

The MWIS of a tree $G = (V, E)$ can be computed exactly with dynamic programming as follows. Let $r \in V$ be the root of G , and for any vertex $v \in V$ let $C(v) \subseteq V$ denote the children of v . Define $A[v]$ as the MWIS of the *subtree rooted at v* where v is fixed to be part of the independent set. Define $B[v]$ analogously but fix v not to be part of the independent set. The MWIS is then $\max\{A[r], B[r]\}$. The recursion carries out as follows:

$$\begin{aligned} A[v] &= w(v) + \sum_{u \in C(v)} B[u] \\ B[v] &= 0 + \sum_{u \in C(v)} \max\{A[u], B[u]\} \end{aligned}$$

(Intuitively, fixing v to be part of the solution forces the children not to be part of the solution. Fixing v not to be in the solution has no constraints on the children. Notice the summation term does not exist for leaves so they serve as base cases)

There are $2n$ subproblems to solve, and each subproblem compares sums up many cases as the number of neighbors of the associated vertex. This already bounds the running time at $\mathcal{O}(n^2)$, but notice that every subproblem associated with a non-root vertex appears at most

twice as a summand (e.g., if v is u 's parent, $A[u]$ is only used when calculating $B[v]$ and $B[u]$ is used only for $A[v]$ and $B[v]$). Thus the total number of additions/comparisons is also bounded linearly and the total running time is $\mathcal{O}(n)$.

2.3 Sparsifying the graph

For some word w , define its candidates $K_w = \{k : T_k \subseteq I_w\}$ as the candidates that propose deleting rows of I_w . We define the *preferred* candidate $k_w \in K_w$ to be the one with the highest coverage (ties broken in favor of larger a_k)². Let $U' \subseteq U$ be the universe restricted to preferred candidates, and consider the arising conflict graph G' .

There are no conflicts between candidates targeting the same word, because the graph has been restricted to a single candidate targeting each word. So the only conflicts on this graph are arising from source-target intersections. Consider directing the edges of the graph such that there is an edge $k \rightarrow k'$ if the source of k intersects the target of k' . Under this orientation, every vertex has at most one outgoing edge: Any two candidates whose targets intersect with the source of k must be targeting the same word (due to the disjointness of LCP intervals), but there can only be candidate targeting each word. The resulting graph is therefore a *pseudoforest*: Every component with t vertices has exactly t edges.

2.4 MWIS on pseudoforests

A pseudoforest consists of *pseudotrees*, which are trees with an additional edge (result in exactly one cycle). The MWIS algorithm for trees can be adapted for pseudotrees by fixing a cycle node and branching on its inclusion in the independent set. Specifically, let P be a pseudotree and $v \in V(P)$ a cycle node, and let $N[v]$ denote the neighbors of v . There are two cases:

- If v is in the solution: $\text{MWIS}(P) = w(v) + \text{MWIS}(P - (\{v\} \cup N[v]))$
- If v is not in the solution: $\text{MWIS}(P) = 0 + \text{MWIS}(P - \{v\})$.

Notice that removing v is sufficient to destroy the only cycle, so the remaining subproblems of either case can be solved exactly in $\mathcal{O}(n)$ time. The overall running time is therefore $\mathcal{O}(n)$.

2.5 Leftover phase

After one pass of the DP algorithm on the preferred candidates, some words end up with no accepted candidates because their preferred candidate was not part of the DP solution. Such words may have had other (non-preferred) candidates that do not conflict with the DP solution, but these candidates were eliminated in the sparsification phase and therefore were not considered yet. These candidates are accounted for with a singular greedy pass. After the DP solution is finalized, all candidates are revisited in decreasing order of coverage and added to the solution if they do not conflict with what is already in the solution.

2.6 Phase 2

The leftover phase is followed by a final pass of **tree-dp**'s phase II procedure (explained in [tree_dp.pdf](#)).

²The preference rule is a heuristic aiming to delete the largest number of rows per word. Different rules may provide better results.

3 Further ideas

Here are some ideas not yet implemented, but could be worth pursuing.

3.1 Combining `pseudoforest-dp` with `tree-dp3`

All DP algorithms in the repository follow the same formula of DP $\rightarrow$ leftover $\rightarrow$ phase 2 with the following differences:

- `tree-dp` generates the same candidate graph but ignores any candidate that would close a cycle. Note that the ignored candidate is decided by the order in which the candidates are processed.
- `tree-dp4` avoids cycles by dropping the lowest coverage candidate of the cycle instead.
- `pseudoforest-dp` allows cycle formation and solves them exactly.
- `tree-dp3` avoids cycles like `tree-dp` and phase II with a different heuristic: It creates a dependency graph on *words* (rather than candidates) and exactly solves small neighborhoods on this graph.

The compression achieved by `pseudoforest-dp`'s DP phase is provably no worse than the other algorithms' (`pseudoforest-dp` sometimes sees more candidates, and never less). However, `tree-dp3` sometimes achieves the overall best compression with its LNS heuristic. This heuristic can also replace phase II in `pseudoforest-dp` to potentially get better results.

3.2 Iterative DP (implemented)

After one pass of the DP algorithm, some words receive no accepted candidates under the optimal DP solution. These words likely have other (non-preferred) candidates which were invisible to the DP: The leftover pass is aimed towards finding and accepting such candidates. Instead of accepting the leftover viable candidates in the greedy order, we can use DP iteratively on the remaining candidates until no further candidates can be generated.

Implemented behind `GCSA_PFD_P_ITERATE=1` (off by default): each further round re-extracts, over the still-unresolved words only, every remaining word's currently *available* best candidate (given what earlier rounds already accepted) and solves that round's pseudoforest exactly again, same as round 0. The loop stops at a fixed point – a round whose DP accepts nothing – or at `GCSA_PFD_P_MAX_ROUNDS` if set; it needs no cap to terminate on its own, since a productive round always accepts at least one more word.

This turned out *not* to be a strict improvement over one DP round followed by the leftover pass, which is worth recording since it runs against the intuition that repeating an exact solve can only help. Each round is exact only against its own single up-front snapshot – one fixed candidate per still-unresolved word, chosen once at the start of that round – and a word whose candidate loses a conflict inside the round cannot fall back to its second-best candidate until an entirely new round is run. The plain greedy leftover pass has no such restriction: it re-derives "the best still-available candidate" one word at a time against always-current state, so it can fall back to a second choice mid-pass in a way no single DP round can. Measured on the `bench/` E. coli genomes across several shapes and `-max-add` values, the end-to-end effect (through leftover, and again through Phase II) is a coin flip: usually no change, otherwise a difference of a handful of $|C|$ positions out of several million kept, in either direction – never large, and never consistently favoring iteration. It is left opt-in pending a wider sweep.

3.3 Source intervals without LCP depth

When generating candidates with some offset a , the algorithms require that the source intervals have $\text{LCP-depth} \geq a + 1$. This is sufficient to ensure that the $a + 1$ -th character of the intervals is the same, but it is not necessary: There can be intervals where the LCP-depth dips below $a + 1$ but the $a + 1$ -th character remains the same: Allowing candidates to source from such intervals can yield better results ³.

3.4 Additional candidates via post-deletion source contiguity

Recall that we require source intervals to be contiguous so that a single reference (`pos`, `num`, `add`) can be used to reconstruct `num` positions on the final structure. More specifically, we want the source rows to be contiguous in the *final* SA: Requiring them to be contiguous on the pre-compression SA is sufficient for this, but not necessary.

As an example, suppose we have a link that sources from the rows $[1, 2, 5, 6]$. This cannot be used to create a valid candidate in our definition, but suppose there was another candidate that targets the rows $[3, 4]$. If we delete these rows, $[1, 2, 5, 6]$ is now contiguous and can be used by a candidate.

³Dropping the LCP-depth requirement breaks the guarantee that the reconstructed positions appear in the same order in the SA as the original positions. But this does not seem to be a requirement for our application.